자료구조 · 정렬 알고리즘

Bubble Sort

옆 칸이랑 비교해서, 큰 값을 뒤로 밀어내는 정렬



핵심 아이디어

이웃한 두 칸을 비교해서,
앞이 더 크면 자리를 바꾼다.

큰 값이 **거품(bubble)**처럼 한 칸씩 끝으로 떠오른다!

ALGORITHM

한 번의 pass에서 하는 일

01

비교

맨 앞부터 **이웃한 두 원소**를 차례로 비교

02

교환

앞의 값이 **더 크면 swap**, 아니면 그대로

03

확정

pass가 끝나면 **가장 큰 값이 맨 끝**에 자리 잡음

→ 이 pass를 **n − 1번** 반복하면 정렬 완료. k번째 pass에서 k번째로 큰 값이 확정!

PROPERTIES

버블소트의 세 가지 성질

In-place	추가 배열 없이 원래 배열 안에서 자리만 바꿈
Stable	앞이 **더 클 때만** swap → 같은 key끼리는 순서가 안 바뀜
Comparison	이웃한 두 key의 크기를 비교해서 정렬

예제 · PASS 1

가장 큰 42가 맨 끝까지 올라간다



예제 · PASS 2

두 번째로 큰 32가 자리를 찾는다



파랑 swap · **테두리** 비교만 · **초록** 확정

예제 · PASS 3 – 5

pass마다 뒤에서부터 하나씩 확정

시작	42	26	19	5	32	13
PASS 1 후	26	19	5	32	13	42
PASS 2 후	19	5	26	13	32	42
PASS 3 후	5	19	13	26	32	42
PASS 4 후	5	13	19	26	32	42
PASS 5 후	5	13	19	26	32	42

TIME COMPLEXITY · 평균 / 최악

O(n²)

pass 1 — **n − 1번 비교**

pass 2 — **n − 2번**

pass 3 — **n − 3번**

⋮

마지막 — **1번**

비교 횟수를 다 더하면

$(n-1) + (n-2) + \dots + 1$

$$= n(n-1)/2 \rightarrow O(n^2)$$

swap도 최악엔 $O(n^2)$번 → **n < 100** 정도 작은 데이터에만 쓸 만함

TIME COMPLEXITY · 최선

이미 정렬돼 있으면 O(n), 단 flag가 있을 때만

```
void bubbleSort(int *arr, int n) {
  for (int i = 0; i < n - 1; i++) {
    int swapped = 0;
    for (int j = 0; j < n - 1 - i; j++) {
      if (arr[j] > arr[j + 1]) {
        int tmp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = tmp;
        swapped = 1;
      }
    }
    if (!swapped) break;
  }
}
```

정렬된 배열이라도 **그냥 두면** 끝까지 다 돌아서 $O(n^2)$

한 pass에서 **swap이 한 번도 없으면** 이미 정렬된 것

flag로 바로 종료 → O(n)

SUMMARY

버블소트 한눈에 보기

항목	내용
방식	이웃한 두 값을 비교 → 앞이 크면 swap, n − 1번 pass
최선	**O(n)** (이미 정렬 + flag)
평균 / 최악	$O(n^2)$
성질	In-place · Stable · Comparison
한 줄 요약	**가장 단순하지만 느림.** 작은 데이터에만!